

MLA - TP - Problème de localisation avec contraintes de design et demande multiple

Ronan Le Prat

Mars 2026

1 Introduction

Dans ce rapport de TP, on s'intéresse au problème de localisation dans lequel :

- On se restreint à un unique client,
- La demande d du client est multiple,
- On a des contraintes de design, sous la forme d'implications ("si un tel site est ouvert, alors on doit ouvrir tel autre").

On résout ce problème via 4 méthodes différentes :

1. **[FC]** - Une formulation compacte du problème, sans décomposition.
2. **[FBPL]** - Résolution par génération de contraintes, où le sous-problème est résolu par programmation linéaire.
3. **[FBEXP]** - Résolution par génération de contraintes, où le sous-problème est résolu à partir d'une expression explicite.
4. **[FBCPLEX]** - Résolution par génération de contraintes, gérée automatiquement par CPLEX.

L'ensemble des formulations formelles des problèmes, proposées par M. Porumbel, est accessible ici :

<https://cedric.cnam.fr/~porumbed/mla/mla.pdf>.

Seule la méthode 1 renvoie l'affectation optimale du client aux sites ouvert. Remarquons qu'on peut néanmoins la déduire facilement connaissant les sites ouverts, en assignant le client aux d premiers sites ouverts dont le coût d'affectation est minimal.

Les PL et PLNE sont résolus en utilisant le solver CPLEX, version 22.1.1.0.

La décomposition de Benders est implémentée naïvement (le problème principal est résolu à l'optimum à chaque itération), et ne fait pas usage des fonctionnalités de callbacks des solvers.

L'ensemble du code utilisé est disponible publiquement dans le dépôt Github suivant :

<https://github.com/Renelle29/MLA-MPRO/tree/main/TP>.

2 Résultats

2.1 Sans contraintes de design supplémentaires

On compare les quatre méthodes sur des instances de taille $n \in [5, 10^6]$ générées par la fonction `loadData`. On conserve également les deux contraintes de design $y_1 \geq y_2$, et $y_1 \geq y_3$.

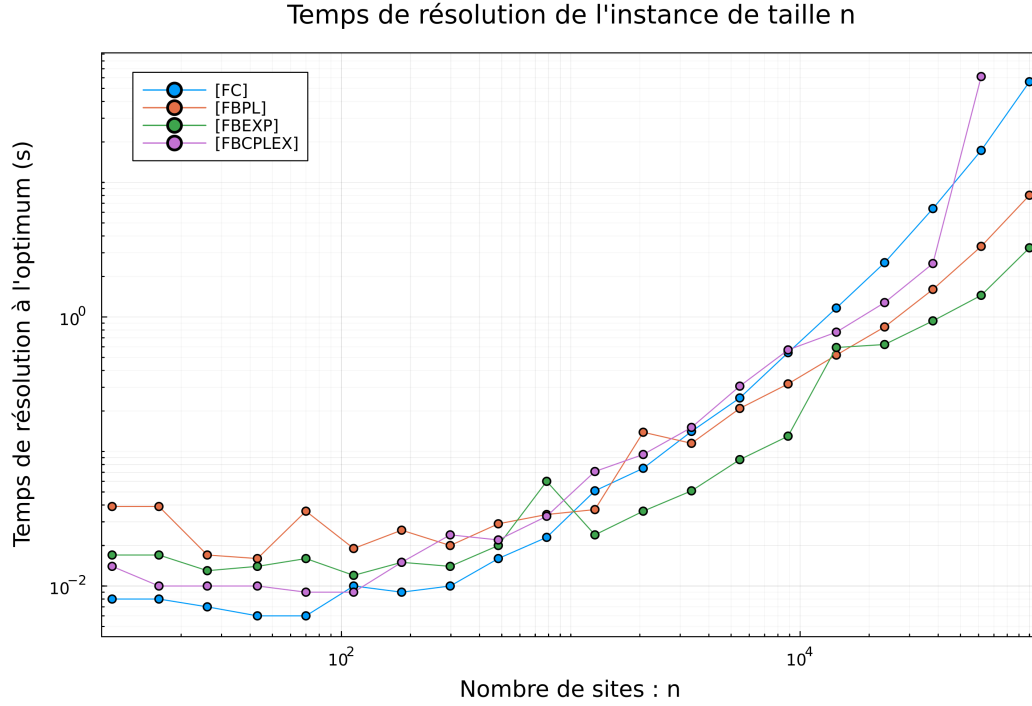


Figure 1: *Comparaison des méthodes de résolution pour le problème sans contraintes de design supplémentaires (échelle log-log).*

Ce que l'on peut constater sur cette figure :

- En-dessous de $n = 3000$, la version compacte est plus rapide. Passé $n = 3000$, la reformulation de Benders montre son intérêt, et permet de résoudre plus rapidement le problème que la version compacte.
- La reformulation de Benders réalisé automatiquement par CPLEX présente peu d'intérêts. Elle se comporte moins bien que [FB] pour $n \leq 3000$, et moins bien que mes implémentations pour $n \geq 3000$.
- La méthode [FBEXP] est presque systématiquement meilleure que [FBPL], ce qui montre que la résolution rapide du problème de séparation permet de résoudre significativement plus rapidement le problème dans ce cadre.

2.2 Avec contraintes de design supplémentaires

Une question que je me suis posée durant le TP, est de savoir à quel point ce problème est difficile.

Il est clair que sans contraintes de design, le problème est trivialement polynomial. On peut trier les $f_i + c_i$ par ordre croissant, et choisir les d premiers sites (ce qui s'effectue en $O(n \log(n))$).

En revanche, lorsque l'on ajoute des contraintes de design de type "implications" (c'est-à-dire des couples (i, j) pour lesquels on impose $y_i \leq y_j$), alors le problème devient NP-Complet. On peut effectivement réduire le problème de Clique Maximum dans un graphe non-orientée à ce problème (réduction proposée par Maël Verbois durant la séance).

Pour cette raison, je propose "d'augmenter" la difficulté du problème, en ajoutant un certains nombre de contraintes de design générées aléatoirement, afin d'obtenir des comparaisons de l'efficacité des méthodes plus intéressantes. Dans la suite, on considérera des instances contenant autant de contraintes de design, que de sites.

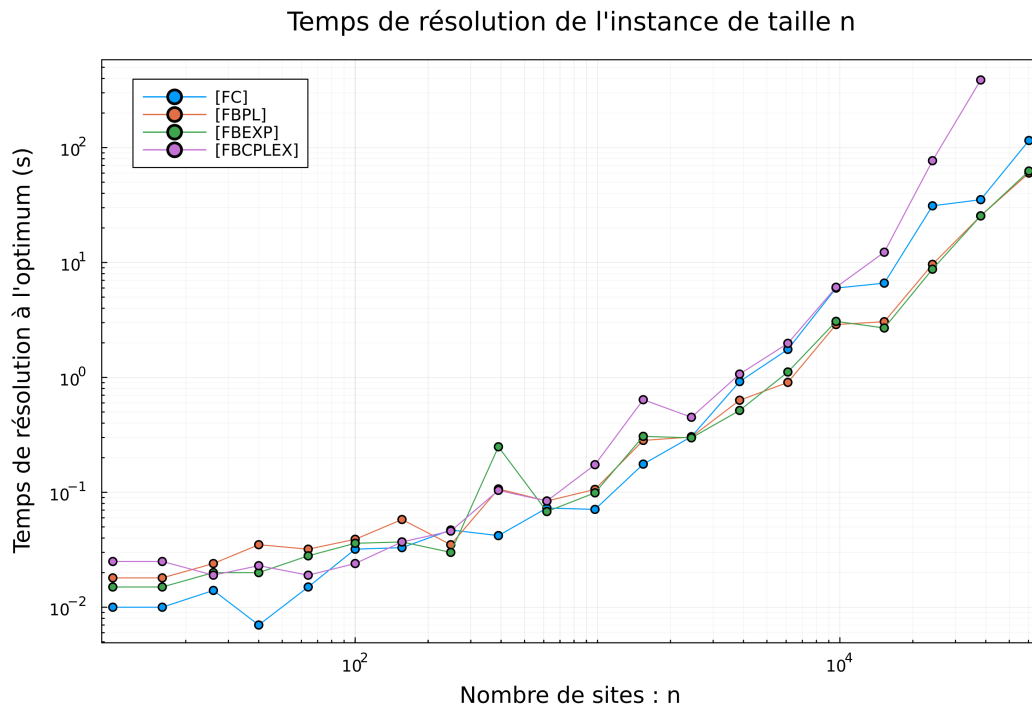


Figure 2: Comparaison des méthodes de résolution pour le problème avec contraintes de design supplémentaires (échelle log-log).

Ce que l'on peut constater sur cette figure :

- Une nouvelle fois, la méthode **[FBCPLEX]** montre rapidement ses limites, à tel point que pour la dernière valeur de n testée (60256), la méthode n'arrive pas à converger en moins de 10 minutes, là où les autres méthodes convergent en moins de 2 minutes. Visiblement cette méthode n'est pas encore au point.
- La méthode **[FBEXP]** semble de nouveau *légèrement* meilleure que **[FBPL]**, même si l'amélioration semble désormais négligeable.

Comparons plus en détail les méthodes **FC** et **FBEXP** :

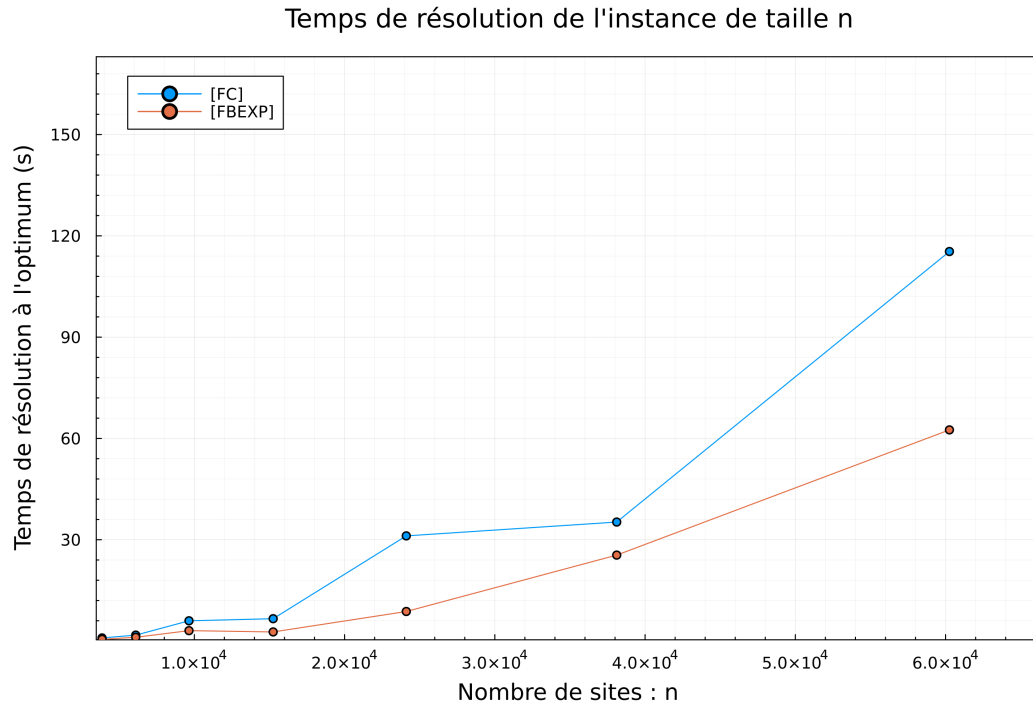


Figure 3: *Comparaison [FC] vs [FBEXP] pour $n \geq 3000$ (échelle lin-lin).*

Ce dernier graphique illustre bien l'intérêt d'une reformulation de Benders pour des instances de grande taille. La plus grande instance ($n = 60256$) est résolue en 62.5s par **[FBEXP]** contre 115.5s par **[FC]**.

3 Annexe

n	optimum	[FC]	[FBPL]	[FBEXP]	[FBCPLEX]
10	222.0	0.008	0.039	0.017	0.014
16	427.0	0.008	0.039	0.017	0.01
26	642.0	0.007	0.017	0.013	0.01
43	1008.0	0.006	0.016	0.014	0.01
70	1825.0	0.006	0.036	0.016	0.009
113	2853.0	0.01	0.019	0.012	0.009
183	4714.0	0.009	0.026	0.015	0.015
298	7945.0	0.01	0.02	0.014	0.024
483	12680.0	0.016	0.029	0.02	0.022
785	20659.0	0.023	0.034	0.06	0.033
1274	33741.0	0.051	0.037	0.024	0.071
2069	54820.0	0.075	0.139	0.036	0.095
3360	89087.0	0.141	0.115	0.051	0.151
5456	144802.0	0.25	0.209	0.087	0.306
8859	235022.0	0.543	0.318	0.13	0.569
14384	381497.0	1.165	0.521	0.593	0.771
23357	619592.0	2.533	0.843	0.624	1.279
37927	1.006077×10^6	6.39	1.603	0.935	2.496
61585	1.633707×10^6	17.296	3.348	1.448	61.24
100000	2.652926×10^6	56.049	8.033	3.266	—

Table 1: Comparaison du temps de résolution en secondes pour différentes méthodes, sans contraintes de design supplémentaires.

n	optimum	solve_time1	solve_time2	solve_time3	solve_time4
10	314.0	0.01	0.018	0.015	0.025
16	556.0	0.01	0.018	0.015	0.025
26	792.0	0.014	0.024	0.02	0.019
40	1289.0	0.007	0.035	0.02	0.023
64	2213.0	0.015	0.032	0.028	0.019
100	3545.0	0.032	0.039	0.036	0.024
156	5697.0	0.033	0.058	0.037	0.037
248	8368.0	0.047	0.035	0.03	0.046
390	12942.0	0.042	0.107	0.249	0.104
618	20665.0	0.073	0.084	0.068	0.084
976	33877.0	0.071	0.106	0.099	0.174
1544	52890.0	0.176	0.283	0.307	0.64
2440	82520.0	0.305	0.303	0.298	0.45
3858	131330.0	0.92	0.633	0.516	1.069
6100	210662.0	1.754	0.905	1.115	1.98
9644	331551.0	5.992	2.886	3.076	6.094
15246	522557.0	6.617	3.054	2.692	12.296
24106	825502.0	31.143	9.639	8.751	76.958
38112	$1.311417e6$	35.239	25.471	25.452	387.812
60256	$2.082577e6$	115.36	60.076	62.532	—

Table 2: Comparaison du temps de résolution en secondes pour différentes méthodes, avec contraintes de design supplémentaires.